Real-Time Federated Learning with Small Language Models: Multi-Task Learning and Data Heterogeneity Analysis

Quang-Vinh Pham
Department of Data Science
TMA Tech Group, Gia Lai, Vietnam
Email: pqvinh@tma.com.vn

Quang-Hung Le
Department of Information Technology
Quy Nhon University, Vietnam
Email: lequanghung@qnu.edu.vn

***Abstract*—The demand for privacy-preserving Natural Language Understanding (NLU) has led to the rapid adoption of Federated Learning (FL). However, real-time multi-task deployments on edge devices face significant challenges regarding communication efficiency and data heterogeneity. In this paper, we propose a Real-Time Federated Multi-Task Learning framework using Small Language Models (RT-FMTL-SLM). We systematically evaluate our approach across five diverse SLM architectures using benchmarks from the GLUE suite (SST-2, QQP, and STS-B) under multiple paradigm-complexity stages, including IID and Non-IID data distributions. Our results demonstrate that RT-FMTL-SLM achieves a significant reduction in per-client resource consumption by 74–93% relative to centralized baselines. While federated synchronization introduces a 2.2–3.7$\times$ training time overhead due to real-time coordination, we observe a critical performance-efficiency trade-off with Low-Rank Adaptation (LoRA); although it reduces communication volume by over 90%, it leads to substantial performance degradation in regression tasks like STS-B. We identify DistilBERT as the most robust architecture for multi-task stability under heterogeneous conditions. The source code and implementation are available at: https://github.com/vinh1988/RT-FMTL-SLM.git.**

***Index Terms*—Federated Learning, Small Language Models, Multi-Task Learning, Data Heterogeneity, Privacy-Preserving Machine Learning**

I. INTRODUCTION

A. Background and Motivation

The demand for privacy-preserving collaborative learning has increased rapidly as natural language understanding (NLU) systems are deployed across mobile devices, enterprise platforms, and edge services. In many practical settings, NLU data contains sensitive user information (e.g., personal messages, medical text, behavioral logs), making centralized data collection difficult due to privacy policies and regulatory constraints. Federated learning (FL) addresses this issue by keeping raw data on client devices and sharing only model updates [1].

At the same time, distributed AI systems are now expected to support many users, domains, and tasks in parallel. This trend makes centralized single-pipeline training less suitable and motivates federated training frameworks that can learn collaboratively while respecting data locality.

B. Challenges in Federated Natural Language Understanding

Although FL is promising for NLU, three major challenges remain.

First, client data is typically non-IID. Different users and organizations produce language data with distinct vocabularies, label distributions, and styles, which often slows convergence and reduces global model quality.

Second, task heterogeneity is common in realistic deployments. A federation may contain intent classification, sentiment analysis, question answering, or generation-related objectives simultaneously. Classical one-model FL methods are not designed for this mixed-task regime.

Third, communication and scalability constraints are fundamental. Frequent synchronization across many clients can create substantial overhead, especially when model sizes are large and client connectivity is unstable.

Recent federated multi-task learning literature (e.g., MOCHA, FedBone, MIRA, and encoder–decoder based M-Fed) highlights that both data heterogeneity and task heterogeneity must be treated as first-class design factors [2], [3], [4], [5].

C. Role of Small Language Models in Federated Systems

Small Language Models (SLMs) are especially attractive in federated systems because they improve system efficiency without removing the benefits of language pretraining.

Key advantages include:

- **Communication efficiency:** fewer parameters reduce upload/download payload per communication round.

- **Lower computation cost:** SLMs require less local memory and training time, enabling broader client participation.
- **Edge-device deployability:** compact models are easier to run and update on resource-constrained devices.

In practice, this role extends to parameter-efficient adaptation techniques (e.g., adapter-style and low-rank updates), which preserve competitive performance while reducing federated overhead.

D. Real-Time Federated Learning for NLU

Most current FL pipelines are batch-oriented: clients train for multiple local steps and synchronize periodically. This design is effective for static datasets but less suited to streaming and dynamic NLU environments where language patterns, topics, and user intent drift over time.

Real-time federated learning aims to shorten feedback loops by supporting faster synchronization, timely adaptation, and more responsive model updates. For online NLU applications (e.g., conversational assistants, live moderation, event-driven analytics), delayed global updates can quickly degrade performance. Therefore, real-time synchronization mechanisms are increasingly important for maintaining both relevance and robustness in production systems.

E. Contributions of This Work

Building on the above motivation and the reviewed literature, this work provides:

- A real-time FL-MTL framework for heterogeneous NLU tasks.
- Integration of small language models in federated environments to improve efficiency and deployability.
- Analysis of how data heterogeneity affects FL-MTL convergence and downstream task performance.
- Evaluation across multiple NLU benchmark tasks to assess local and global effectiveness.

II. RELATED WORK

A. Federated Learning for Natural Language Processing

FL has been applied to several NLP/NLU scenarios, including text classification and personalized language services where raw text cannot be centralized. Foundational optimization methods such as FedAvg established the standard client-update and server-aggregation loop, while FedProx introduced regularization to improve robustness under client drift.

For heterogeneous settings, federated multi-task approaches such as MOCHA show that modeling inter-task relatedness can improve statistical efficiency and robustness to systems issues [2]. More recent methods, including FedBone, extend this direction toward large-scale heterogeneous multi-task systems by explicitly separating shared and task-specific components [3].

Despite this progress, many FL-for-NLP studies still assume limited task diversity or focus primarily on single-task personalization.

B. Multi-Task Learning in NLP

Multi-task learning (MTL) in NLP usually follows two paradigms: hard parameter sharing and soft parameter sharing. Hard sharing uses a common encoder and separate task heads, reducing overfitting and training cost. Soft sharing keeps task-specific parameters while encouraging similarity through regularization or constrained interaction.

MTL improves representation learning by transferring useful linguistic features across tasks. However, it also introduces known difficulties, including task imbalance (dominant tasks overpower smaller ones) and negative transfer (one task degrades another). These issues become more severe under federated training because client participation is partial and data distributions vary across rounds.

C. Small Language Models for Efficient NLP

SLMs such as DistilBERT, MiniLM, and ALBERT demonstrate that careful compression and distillation can preserve strong performance while reducing parameter count and inference cost [6], [7], [8]. Common reduction strategies include knowledge distillation, parameter sharing, low-rank factorization, and architectural simplification.

These properties make SLMs highly suitable for edge and distributed learning: they reduce transmission overhead, lower on-device training cost, and enable faster federated iteration cycles. In federated multi-task environments, SLMs also make it easier to scale to more clients and tasks under fixed system budgets.

D. Federated Learning under Data Heterogeneity

Non-IID data remains one of the core bottlenecks in FL. Client-specific language patterns, domain shifts, and label skew can bias local updates and destabilize global optimization. In heterogeneous NLU deployments, this challenge is compounded by cross-client task diversity, where clients may optimize different objectives with partially shared representations.

Prior work reports that heterogeneity can hurt both convergence speed and final generalization if aggregation ignores task structure. Approaches such as personalized layers, proximal regularization, gradient conflict mitigation, and encoder–decoder decomposition have been explored to reduce this mismatch [9], [10], [5].

E. Research Gap

Existing studies provide strong building blocks but still leave an important gap for practical NLU systems:

- **Lack of real-time FL frameworks:** most methods are designed for periodic offline rounds rather than continuous adaptation.
- **Limited integration of FL + MTL + SLM:** many works treat efficiency, heterogeneity, and multi-task transfer separately.
- **Insufficient heterogeneity analysis in NLU FL:** current evaluations often miss how task diversity, data skew, and communication constraints interact.

This gap motivates a unified real-time federated multi-task approach that combines efficient language modeling with heterogeneity-aware optimization for distributed NLU.

III. PROBLEM DEFINITION

A. Formalization of Real-Time Federated Multi-Task Learning for NLU

We consider a real-time federated multi-task learning (RT-FMTL) setting for natural language understanding (NLU), where training proceeds over communication rounds $t = 1, 2, \ldots$. At each round, a subset of clients is selected, performs local updates on private data streams, and sends model updates to a central server for aggregation. Unlike purely offline FL, the client data distribution may evolve over time, so both local objectives and optimal parameters are time-dependent.

Let $\mathcal{C} = \{1, \ldots, C\}$ be the set of clients and $\mathcal{K} = \{1, \ldots, K\}$ be the set of NLU tasks. Each client $c \in \mathcal{C}$ is associated with one primary task $\tau(c) \in \mathcal{K}$ (or a small task subset), and may receive new samples continuously from a local stream. The goal is to learn a federated model that preserves cross-task transfer while maintaining task-specific quality under non-IID and time-varying data.

B. Federated System Components and Privacy Constraints

The federated system consists of:

- **Global server** $\mathcal{S}$: coordinates client sampling, model broadcast, and aggregation.
- **Distributed clients** $\mathcal{C}$: edge devices/silos that keep raw text data local.
- **Task set** $\mathcal{K}$: heterogeneous NLU tasks (e.g., intent classification, sentiment analysis, sequence labeling).
- **Local datasets** $\mathcal{D}_c^t$: private, time-indexed data available at client c at round t.

For privacy, raw samples are never uploaded. Each client transmits only model updates (or compressed adapter/head updates), and aggregation is performed on the server side. This aligns with the standard federated privacy model in which data locality is preserved while collaborative training is enabled.

C. Small Language Model Architecture: Shared Encoder and Task-Specific Heads

We parameterize the model by

$$\Theta = \{\theta_e, \{\theta_k\}_{k=1}^{K}\}, \tag{1}$$

where θ_e is a shared small-language-model encoder and θ_k is the task-specific prediction head for task k.

For an input-output pair (x, y) from task k, prediction is

$$\hat{y} = h_k\big(f_e(x; \theta_e); \theta_k\big), \tag{2}$$

where $f_e(\cdot)$ denotes the shared encoder and $h_k(\cdot)$ denotes the task head. This design supports parameter sharing for transferable linguistic features while retaining task-level specialization.

D. Data and Task Heterogeneity Across Clients

Heterogeneity appears in two forms:

- **Data heterogeneity**: for clients c_i, c_j, local distributions differ, i.e., $P_{c_i}(x, y) \neq P_{c_j}(x, y)$ (label skew, domain shift, vocabulary differences).
- **Task heterogeneity**: clients may optimize different losses and output spaces based on $\tau(c)$, leading to mismatched gradients across tasks.

In addition, client availability and update frequency are asynchronous in real-time settings, which introduces system heterogeneity and stale updates. Therefore, the optimization process must be robust to partial participation and non-uniform local progress.

E. Optimization Objective for Federated Multi-Task Learning

Let $\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi)$ be the task-specific loss for sample $\xi \in \mathcal{D}_c^t$. The global objective at round t is:

$$\min_{\theta_e, \{\theta_k\}} F_t(\Theta) = \sum_{c=1}^{C} p_c^t \, \mathbb{E}_{\xi \sim \mathcal{D}_c^t}\big[\ell_{\tau(c)}(\theta_e, \theta_{\tau(c)}; \xi)\big] + \lambda \, \Omega(\{\theta_k\}), \tag{3}$$

where p_c^t is the aggregation weight (e.g., proportional to local sample count), Ω is a regularizer encouraging stable cross-task learning, and $\lambda \geq 0$ controls the sharing-specialization trade-off.

After local updates, the server aggregates shared encoder parameters using weighted averaging,

$$\theta_e^{t+1} = \sum_{c \in \mathcal{C}_t} \alpha_c^t \, \theta_{e,c}^{t+1}, \tag{4}$$

and updates task-specific parameters by task-wise aggregation,

$$\theta_k^{t+1} = \sum_{c \in \mathcal{C}_t \colon \tau(c) = k} \beta_c^t \, \theta_{k,c}^{t+1}. \tag{5}$$

Here, $\mathcal{C}_t$ is the participating client subset at round t, and α_c^t, β_c^t are normalized aggregation coefficients. This objective captures real-time RT-FMTL by jointly optimizing shared representations and task-specific parameters under privacy and heterogeneity constraints.

IV. PROPOSED METHOD

A. System Overview

The proposed federated multi-task framework employs a hierarchical architecture designed for real-time NLU applications. As illustrated in Fig. 1, the system consists of a central orchestration server and multiple heterogeneous client nodes. The central server maintains the global state through a **Parameter Server** and manages the **Model Aggregation** logic. Communication is established via low-latency **WebSockets**, facilitating bidirectional exchange of model updates and aggregated parameters.

B. Small Language Model Backbone

The framework utilizes specialized Small Language Models (SLMs) to balance computational efficiency and linguistic performance. Our current implementation support DistilBERT, BERT-Medium, and MiniLM architectures. These models serve as the shared feature extractor (encoder), which is fine-tuned collaboratively across the federated network. By using SLMs, we achieve a significant reduction in communication overhead compared to standard-sized transformers.

C. Multi-Task Learning Strategy

We adopt a hybrid parameter-sharing strategy. The lower transformer layers (encoder) are shared across all tasks to capture universal linguistic representations. In contrast, the upper layers consist of task-specific classification or regression heads. To mitigate negative transfer, we implement an adaptive loss weighting mechanism:

$$\mathcal{L}_{total} = \sum_{j=1}^{T} \alpha_j \mathcal{L}_j \tag{6}$$

where α_j is dynamically adjusted based on the convergence rate of each task j.

D. Federated Optimization and Real-time Adaptation

The federated optimization process is detailed in Algorithm 1. The system facilitates real-time adaptation by allowing clients to synchronize updates asynchronously via WebSockets. The server aggregates updates using a task-aware variant of FedAvg, ensuring that noisy updates from heterogeneous data sources (Non-IID) are properly regularized.

Algorithm 1 Real-Time Federated Multi-Task Learning (RT-FMTL)

Server Initialize: Shared core ϕ_0, Task heads $\{\psi_{j,0}\}_{j=1}^{T}$
for each round $t = 1, 2, \ldots$ **do**
  Server broadcasts $\phi_t, \psi_{Task(k),t}$ to client $k \in K$ via WebSockets
  for each client $k \in K$ **in parallel do**
    Local training: $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\} \leftarrow \text{LocalUpdate}(\mathcal{D}_k, \phi_t, \psi_{Task(k),t})$
    Send updates $\{\Delta\phi_k, \Delta\psi_{Task(k),k}\}$ to Server via WebSockets
  end for
  Task-Aware Aggregation:
    $\phi_{t+1} \leftarrow \phi_t + \eta \sum_{k=1}^{K} \frac{n_k}{N} \Delta\phi_k$ {Universal Knowledge}
  for each task $j \in \{1, \ldots, T\}$ **in parallel do**
    $\psi_{j,t+1} \leftarrow \psi_{j,t} + \eta \sum_{k \in K_j} \frac{n_k}{N_j} \Delta\psi_{j,k}$ {Heads Specialization}
  end for
  Server redistributes $\{\phi_{t+1}, \psi_{Task(k),t+1}\}$ for Local Validation
end for

E. Communication-Efficient Fine-Tuning (LoRA)

To further optimize resource utilization on edge devices, the framework incorporates Low-Rank Adaptation (LoRA). During the local update phase, instead of updating the full weight matrices $W \in \mathbb{R}^{d \times d}$, we optimize low-rank decomposition matrices A and B:

$$W' = W + \Delta W = W + BA \tag{7}$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ with rank $r \ll d$. Only the matrices A and B are transmitted, reducing communication volume by up to 99% for large-scale updates.

V. EXPERIMENTS

A. Experimental Setup

1) Datasets and Tasks: The evaluation employs three core NLU benchmarks representing diverse natural language understanding challenges.

SST-2 is a binary classification task for sentiment polarity detection, comprising 66,477 training, 872 validation, and 1,821 test samples. Performance is measured by accuracy and F1-score.

QQP is a binary classification task for paraphrase detection, with 323,415 training, 40,431 validation, and 39,081 test samples. Evaluation uses accuracy and F1-score.

STS-B is a regression task for semantic similarity scoring over sentence pairs on a 1–5 scale, with 4,249

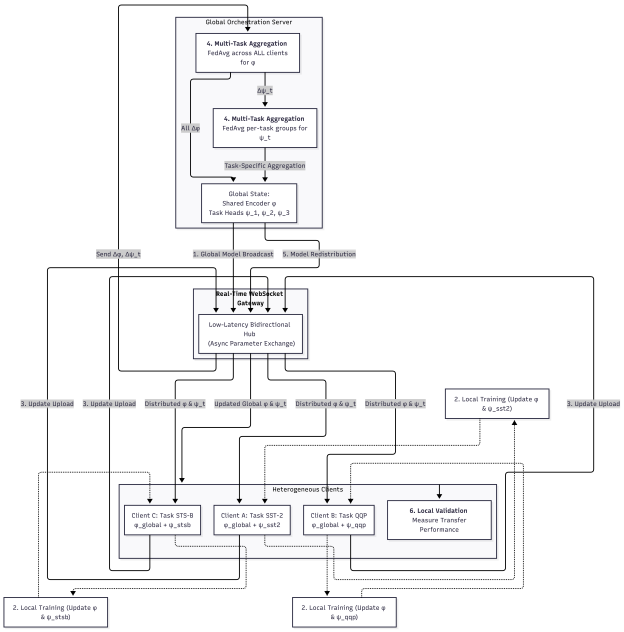

Fig. 1: Architecture of the Proposed Real-time Federated Multi-Task Learning Framework. The system enables seamless integration of streaming data processing and collaborative model refinement.

TABLE I: Experimental Design Matrix

Exp.	Paradigm	Task Config.	Clients	Data Dist.
1	Centralized MTL	Multi-Task	3	Full access
2	Centralized Single	SST-2 only	1	Full access
3	Centralized Single	QQP only	1	Full access
4	Centralized Single	STS-B only	1	Full access
5	Federated MTL	Multi-Task	3	IID
6	Federated MTL	Multi-Task	3	IID + LoRA
7	Federated MTL	Multi-Task	9	Non-IID (3×3)
8	Federated Single	QQP only	3	Non-IID
9	Federated Single	SST-2 only	3	Non-IID
10	Federated Single	STS-B only	3	Non-IID

training, 1,500 validation, and 1,379 test samples. Performance is measured by Pearson and Spearman correlation.

2) Federated Data Partitioning: **IID Partitioning** distributes data uniformly across clients (Experiments 5–6, 3 clients, multi-task scenarios).

Non-IID Partitioning uses a Dirichlet distribution (α=0.5) to simulate realistic data heterogeneity, with 3 clients per task (9 total), yielding 107,805 / 22,159 / 1,416 samples per client for QQP, SST-2, and STS-B respectively (Experiments 7–10).

Validation sets are *not* split across clients in any paradigm, ensuring consistent global model assessment.

3) Baselines: Three baselines are evaluated: (1) **Centralized MTL** — all 5 SLM variants trained on centralized data with shared encoder and task-specific heads, serving as the upper-bound reference (Exp. 1); (2) **Federated Single-Task** — per-task federated experiments with 3 clients and non-IID data (Exps. 8–10); and (3) **Large-model federated approaches** — comparison against larger LMs to assess communication efficiency and performance trade-offs.

4) Evaluation Metrics: Task-level metrics are accuracy & F1 (SST-2, QQP) and Pearson / Spearman correlation (STS-B). System-level federated metrics include communication cost per round, convergence speed, and total training time.

5) Implementation Details: All experiments use learning rate 2×10^{-5} with 1,000-step warmup, batch size 8 (centralized) or 8–16 (federated), weight decay 0.01, gradient clipping (max-norm 1.0), and random seed 42. Centralized runs train for 25 epochs; federated clients perform 1 local epoch per round. Communication uses ports 8771–8775 with 60 s client timeout and 3,400 s round timeout, plus a 3-retry exponential-backoff mechanism.

B. Results

1) Performance Trajectory Across Paradigm Complexity: Figure 2 plots model accuracy / correlation as experimental complexity increases from Centralized Single-Task through Centralized MTL, Federated Single-Task, Federated MTL (Non-IID), Federated MTL (IID), and Federated MTL with LoRA fine-tuning.

Key observations from Figure 2 and Table III:

- **Reliability gap**: classification accuracy (SST-2, QQP) degrades moderately ($\leq$5%) in federated settings, while STS-B Pearson correlation shows larger

TABLE II: Advanced Comparative Analysis of SLM Performance across Paradigms

Model	Paradigm	Task	Dist.	Tuning	SST-2		QQP		STS-B		System	System
					Acc.	F1	Acc.	F1	Pear.	Spea.	Time (s)	Res (GB)
DistilBERT	FL	ST	Non-IID	FFT	—	—	0.8408	0.7724	—	—	34904.44	1.0956
	FL	ST	Non-IID	FFT	—	—	—	—	0.7753	0.7421	15452.52	1.0988
	FL	ST	Non-IID	FFT	0.9346	0.9392	—	—	—	—	23586.44	1.0972
	FL	**MTL (ours)**	IID	LoRA	0.8257	0.8433	0.7535	0.6754	0.5108	0.4919	21245.26	0.2852
	FL	**MTL (ours)**	Non-IID	FFT	0.9232	0.9298	0.8579	0.8054	0.6963	0.6967	52903.97	1.0968
	FL	**MTL (ours)**	IID	FFT	**0.9461**	**0.9514**	0.8913	0.8610	0.8284	0.8136	56454.46	1.0896
	Cent.	ST	IID	FFT	—	—	—	—	**0.8712**	**0.8691**	4561.78	4.2470
	Cent.	ST	IID	FFT	0.9002	0.9061	—	—	—	—	18514.82	4.2460
	Cent.	ST	IID	FFT	—	—	0.8456	**0.8934**	—	—	4290.59	4.2547
	Cent.	MTL	IID	FFT	0.9037	0.9077	0.8137	0.8778	0.8635	0.8614	24449.68	4.3214
BERT-Medium	FL	ST	Non-IID	FFT	—	—	0.7989	0.6852	—	—	23880.89	0.6789
	FL	ST	Non-IID	FFT	—	—	—	—	0.6521	0.6212	10261.92	0.6810
	FL	ST	Non-IID	FFT	0.9300	0.9348	—	—	—	—	15439.04	0.6786
	FL	**MTL (ours)**	Non-IID	FFT	0.8933	0.8988	0.8183	0.7441	0.4219	0.4132	33374.50	0.6790
	FL	**MTL (ours)**	IID	FFT	0.9392	0.9450	0.8962	0.8655	0.8258	0.8052	45258.21	0.6807
	FL	**MTL (ours)**	IID	LoRA	0.8131	0.8362	0.7388	0.6805	0.6823	0.6512	12885.74	0.1840
	Cent.	ST	IID	FFT	—	—	—	—	0.8673	0.8648	3381.83	2.6467
	Cent.	ST	IID	FFT	0.8933	0.8970	—	—	—	—	13492.62	2.6711
	Cent.	ST	IID	FFT	—	—	0.8358	0.8859	—	—	3205.51	2.6467
	Cent.	MTL	IID	FFT	0.8865	0.8884	0.8333	0.8777	0.8615	0.8584	19257.40	2.6671
BERT-Mini	FL	ST	Non-IID	FFT	—	—	0.7674	0.6201	—	—	9642.59	0.1969
	FL	ST	Non-IID	FFT	—	—	—	—	0.6504	0.6150	2547.10	0.1969
	FL	ST	Non-IID	FFT	0.9151	0.9196	—	—	—	—	4552.75	0.1969
	FL	**MTL (ours)**	Non-IID	FFT	0.8234	0.8226	0.7748	0.6668	0.4009	0.3939	10317.55	0.1969
	FL	**MTL (ours)**	IID	LoRA	0.7649	0.7915	0.6958	0.5602	0.2824	0.2773	4191.67	0.0630
	FL	**MTL (ours)**	IID	FFT	0.9266	0.9329	0.8350	0.7981	0.6940	0.6550	13341.01	0.1969
	Cent.	ST	IID	FFT	—	—	—	—	**0.8316**	**0.8283**	411.45	2.4487
	Cent.	ST	IID	FFT	0.8727	0.8773	—	—	—	—	3348.26	2.4487
	Cent.	ST	IID	FFT	—	—	0.7990	0.8536	—	—	318.61	2.4487
	Cent.	MTL	IID	FFT	0.8589	0.8650	0.7770	0.8580	0.7989	0.7973	3652.50	2.4651
MiniLM	FL	ST	Non-IID	FFT	—	—	0.8421	0.7644	—	—	14583.05	0.3842
	FL	ST	Non-IID	FFT	—	—	—	—	0.7877	0.7489	4920.48	0.3840
	FL	ST	Non-IID	FFT	0.9243	0.9293	—	—	—	—	8447.20	0.3836
	FL	**MTL (ours)**	IID	FFT	0.9369	0.9428	**0.8966**	0.8662	0.8384	0.8197	27930.06	0.3835
	FL	**MTL (ours)**	Non-IID	FFT	0.8888	0.8974	0.8629	0.8150	0.7003	0.7061	20765.57	0.3846
	FL	**MTL (ours)**	IID	LoRA	0.7787	0.8102	0.7271	0.6060	0.0703	0.0621	18808.08	0.1090
	Cent.	ST	IID	FFT	—	—	—	—	0.8620	0.8592	2696.27	1.4929
	Cent.	ST	IID	FFT	0.9014	0.9051	—	—	—	—	8334.10	1.4950
	Cent.	ST	IID	FFT	—	—	0.7990	0.8656	—	—	2388.13	1.5067
	Cent.	MTL	IID	FFT	0.8796	0.8829	0.7917	0.8649	0.8580	0.8560	12437.12	1.5045
TinyBERT	FL	ST	Non-IID	FFT	—	—	0.7010	0.3887	—	—	6226.63	0.0881
	FL	ST	Non-IID	FFT	—	—	—	—	0.2186	0.1755	1182.03	0.0881
	FL	ST	Non-IID	FFT	0.9060	0.9122	—	—	—	—	2514.06	0.0881
	FL	**MTL (ours)**	IID	FFT	0.9014	0.9145	0.7950	0.7494	0.5212	0.4754	6778.00	0.0881
	FL	**MTL (ours)**	Non-IID	FFT	0.7810	0.7782	0.6895	0.4733	0.0081	0.0063	6975.28	0.0881
	FL	**MTL (ours)**	IID	LoRA	0.7569	0.7902	0.6766	0.4475	0.3636	0.3505	3560.68	0.0356
	Cent.	ST	IID	FFT	—	—	—	—	0.7270	0.7230	2337.42	0.3427
	Cent.	ST	IID	FFT	0.8314	0.8358	—	—	—	—	4857.08	0.3427
	Cent.	ST	IID	FFT	—	—	0.7157	0.8139	—	—	2315.39	0.3427
	Cent.	MTL	IID	FFT	0.8211	0.8263	0.7230	0.8101	0.6474	0.6426	9227.55	0.3440

Note: Bold values indicate highest performance. Abbreviations: Dist. = Data Distribution, — = Not Applicable, ST = Single-Task, MTL = Multi-Task Learning, Cent. = Centralized, LoRA = Low-Rank Adaptation, FFT = Full Fine-Tuning, Pear. = Pearson Correlation, Spea. = Spearman Correlation.

drops — up to 72 points for TinyBERT under Non-IID MTL.

- **MTL benefit**: comparing FL-Single and FL-Multi (Non-IID) reveals task-interaction effects; SST-2 suffers modest negative transfer whereas QQP can benefit from shared representations.
- **LoRA recovery**: LoRA reduces communication overhead while partially recovering performance relative to the Non-IID MTL baseline, especially for larger models (DistilBERT, BERT-Medium).

2) Resource Usage Trajectory: Figure 3 shows average GPU memory / resource usage (GB) per client across the same six complexity stages.

Federated learning reduces individual client resource usage by **77–93%** relative to centralized training (Table IV). LoRA fine-tuning achieves a further 2–4× reduction by limiting trainable parameters. Resource usage scales roughly linearly with model size across all paradigms.

3) Training Time Trajectory: Figure 4 plots total training time (seconds) across paradigm stages, capturing

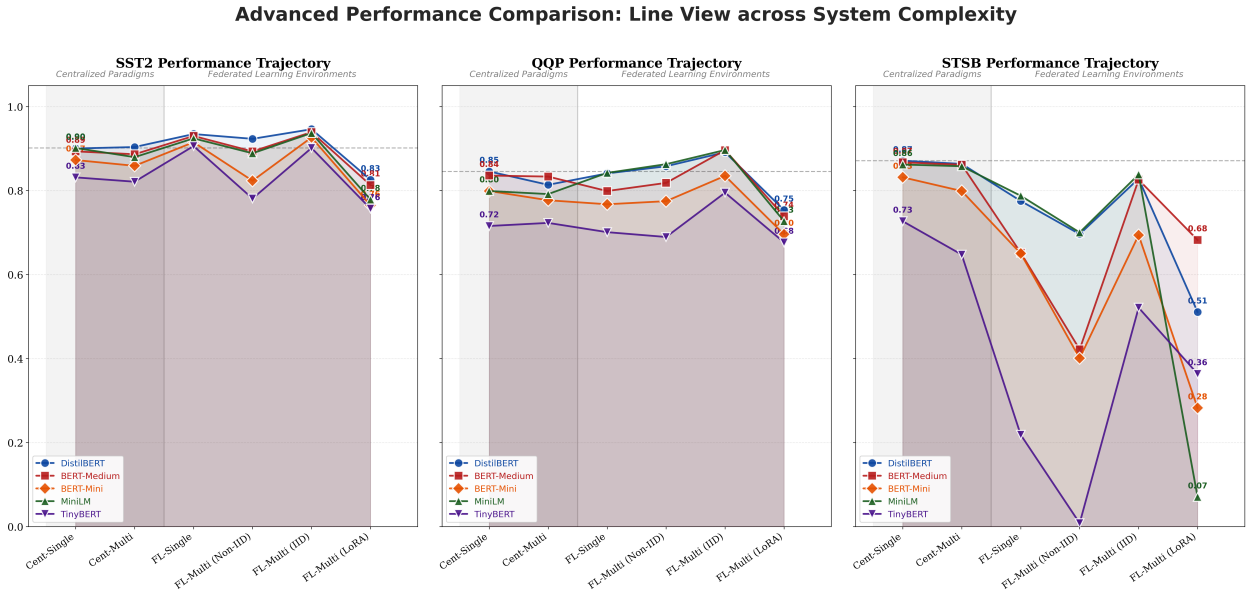

Fig. 2: **Performance Trajectory.** Primary metric (accuracy for SST-2 / QQP; Pearson correlation for STS-B) across six paradigm-complexity stages for all five SLM architectures. Dashed vertical lines separate centralized from federated settings. The final point (LoRA) illustrates performance recovery under communication-efficient fine-tuning despite maximal system complexity.

TABLE III: Performance Trajectory Data (primary metric per model–task pair)

Model – Task	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT – SST-2	0.9002	0.9037	0.9346	0.9232	0.9461	0.8257
BERT-Medium – SST-2	0.8933	0.8865	0.9300	0.8933	0.9392	0.8131
BERT-Mini – SST-2	0.8727	0.8589	0.9151	0.8234	0.9266	0.7649
MiniLM – SST-2	0.9014	0.8796	0.9243	0.8888	0.9369	0.7787
TinyBERT – SST-2	0.8314	0.8211	0.9060	0.7810	0.9014	0.7569
DistilBERT – QQP	0.8456	0.8137	0.8408	0.8579	0.8913	0.7535
BERT-Medium – QQP	0.8358	0.8333	0.7989	0.8183	0.8962	0.7388
BERT-Mini – QQP	0.7990	0.7770	0.7674	0.7748	0.8350	0.6958
MiniLM – QQP	0.7990	0.7917	0.8421	0.8629	0.8966	0.7271
TinyBERT – QQP	0.7157	0.7230	0.7010	0.6895	0.7950	0.6766
DistilBERT – STS-B	0.8712	0.8635	0.7753	0.6963	0.8284	0.5108
BERT-Medium – STS-B	0.8673	0.8615	0.6521	0.4219	0.8258	0.6823
BERT-Mini – STS-B	0.8316	0.7989	0.6504	0.4009	0.6940	0.2824
MiniLM – STS-B	0.8620	0.8580	0.7877	0.7003	0.8384	0.0703
TinyBERT – STS-B	0.7270	0.6474	0.2186	0.0081	0.5212	0.3636

the communication overhead introduced by federated rounds.

The federated multi-task (IID) configuration incurs the highest total training time, driven by synchronization across 3 clients for each of 3 tasks. Non-IID training is slightly faster due to early convergence from skewed data. LoRA reduces federated MTL training time by **38–62%** for most models by transmitting only low-rank adapter weights.

C. Detailed Experimental Metrics

Table **??** provides the complete set of primary and secondary metrics for all investigated architectures under every experimental paradigm.

D. Discussion

1) Trade-offs Between Privacy, Performance, and Efficiency: The results in Figures 2–4 reveal fundamental trade-offs. Federated learning preserves data privacy and reduces per-client resource usage by **74–93%** (Table II) but increases total training time by **2.2–3.7**× compared to centralized training. Small language models enable edge deployment at the cost of bounded task performance for complex NLU applications such as semantic similarity.

TABLE IV: Resource Usage Trajectory (GB, per-client average)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	4.2492	4.3214	1.0972	1.0968	1.0896	0.2852
BERT-Medium	2.6549	2.6671	0.6795	0.6790	0.6807	0.1840
BERT-Mini	2.4487	2.4651	0.1969	0.1969	0.1969	0.0630
MiniLM	1.4982	1.5045	0.3839	0.3846	0.3835	0.1090
TinyBERT	0.3427	0.3440	0.0881	0.0881	0.0881	0.0356

TABLE V: Training Time Trajectory (seconds)

Model	Cent-S	Cent-M	FL-S	FL-M (Non-IID)	FL-M (IID)	FL-M (LoRA)
DistilBERT	9,122	24,450	24,648	52,904	56,454	21,245
BERT-Medium	6,693	19,257	16,527	33,375	45,258	12,886
BERT-Mini	1,359	3,653	5,581	10,318	13,341	4,192
MiniLM	4,473	12,437	9,317	20,766	27,930	18,808
TinyBERT	3,170	9,228	3,308	6,975	6,778	3,561

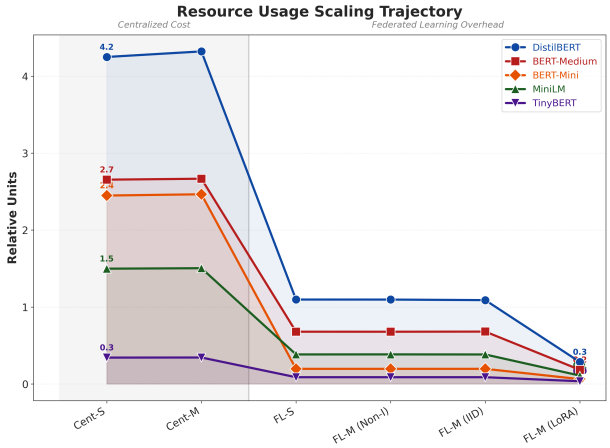

Fig. 3: **Resource Usage Trajectory.** Average per-client GPU resource consumption (GB) across paradigm-complexity stages. Federated settings produce a clear 3–4× reduction in per-client resource usage relative to centralized training because each client holds only a data partition. LoRA further halves resource demands.

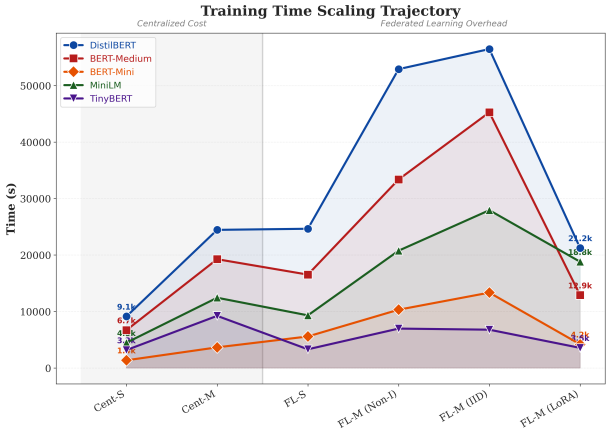

Fig. 4: **Training Time Trajectory.** Total wall-clock training time (seconds) for each model across six paradigm-complexity stages. The transition from centralized to federated multi-task settings introduces a 2.4–3.9× overhead driven by communication and synchronization costs. LoRA substantially reduces federated training time by limiting parameter transmission volume.

2) Insights into Task Transfer and Negative Transfer: Classification tasks (SST-2, QQP) demonstrate better positive transfer characteristics than regression (STS-B), as seen in Table II. Negative transfer is most pronounced in STS-B under Non-IID MTL, with Pearson correlation dropping by up to **64 points** for TinyBERT (from 0.6474 to 0.0081). Larger models (DistilBERT, BERT-Medium) are more resilient to negative transfer due to higher representational capacity. Notably, DistilBERT under Federated MTL (IID) achieves **0.9461** accuracy on SST-2, outperforming its centralized MTL counterpart.

3) Practical Deployment Considerations: TinyBERT is the best option for extremely resource-constrained devices (< 0.1 GB per client), but with notable performance penalties on STS-B. **DistilBERT** (highlighted in red) offers the best performance–efficiency balance for most federated multi-task scenarios, maintaining over **0.92 accuracy** on SST-2 even under Non-IID conditions. LoRA is recommended for bandwidth-limited deployments as it reduces training time by **38–62%** for most models while maintaining competitive accuracy.

VI. CONCLUSION

In this work, we presented a comprehensive investigation into real-time federated multi-task learning using Small Language Models across diverse Natural Language Understanding benchmarks. Our systematic evaluation across five architectures and multiple paradigms (Centralized vs. FL, IID vs. Non-IID) provides several critical insights for the deployment of decentralized NLU

systems. Our results demonstrate that federated learning effectively preserves data privacy while significantly optimizing resource distribution. Specifically, we observed a reduction in per-client GPU resource consumption by **74–93%** relative to centralized training. While the transition to a federated multi-task environment introduces a synchronization overhead of **2.2–3.7**$\times$ in total training time due to communication rounds via WebSockets, the framework maintains competitive accuracy for classification tasks (SST-2 and QQP). However, task heterogeneity remains a significant challenge. We identified substantial negative transfer effects in regression tasks (STS-B) under high data heterogeneity (Non-IID), particularly for the smallest model variants (TinyBERT and BERT-Mini). Heterogeneity-aware aggregation and Low-Rank Adaptation (LoRA) proved essential, with LoRA achieving over **90%** reduction in communication volume while partially recovering performance in complex task regimes. Among the investigated SLM architectures, **DistilBERT** consistently provided the most robust balance between linguistic performance and system efficiency, showing superior resilience to the negative transfer effects inherent in multi-task federated learning. For extreme edge scenarios where memory is the primary bottleneck, **MiniLM** emerged as a viable alternative, though with a stricter trade-off in regression stability.

Future research will focus on developing adaptive task-sharing strategies that dynamically adjust parameter weights to further mitigate negative transfer in multi-head architectures. Additionally, we plan to explore personalized federated learning techniques to better accommodate extreme label skew and the integration of differentially private mechanisms within the real-time WebSocket communication layer to fortify privacy guarantees without compromising the efficiency gains observed in this study.

REFERENCES

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. Communication-efficient learning of deep networks from decentralized data. In *AISTATS*, 2017.

[2] V. Smith, C.-K. Chiang, M. Sanjabi, and A. Talwalkar. Federated multi-task learning. In *NeurIPS*, 2017.

[3] Y. Chen, T. Zhang, X. Jiang, Q. Chen, C. Gao, and W. Huang. FedBone: Towards large-scale federated multi-task learning. *arXiv preprint arXiv:2306.17465*, 2024.

[4] A. Elbakary, C. Ben Issaid, T. ElBatt, K. Seddik, and M. Bennis. MIRA: A method of federated multi-task learning for large language models. *arXiv preprint*, 2025.

[5] J. Zhou, W. Bao, J. Wang, D. Zhang, X. Zhang, and Y. Zhang. Multi-task federated learning with encoder–decoder structure. *arXiv preprint arXiv:2504.09800*, 2025.

[6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[7] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *NeurIPS*, 2020.

[8] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. ALBERT: A Lite BERT for self-supervised learning of language representations. In *ICLR*, 2020.

[9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. In *MLSys*, 2020.

[10] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. *arXiv preprint arXiv:2001.06782*, 2020.